Exercises 5.1–5.12 — Monte Carlo Methods

J.C. Vaught

Spring 2026 — CSCE 775

How Blackjack Works (Quick Primer)

If you have never played blackjack, here is all you need to know for this exercise.

The goal. Get a hand of cards whose values add up as close to 21 as possible, *without going over*. If your total exceeds 21 you "bust" and lose immediately.

Card values. Number cards (2–10) are worth their face value. Face cards (Jack, Queen, King) are each worth 10. An Ace is special: it can count as either 1 or 11, whichever helps you more.

A round of play. Both you (the player) and the dealer start with two cards. You can see one of the dealer's cards (the "showing" card) but not the other. On your turn you have two choices:

- **Hit** — take another card (adds to your total, but risks going over 21).
- **Stick** (also called "stand") — stop taking cards and keep your current total.

After you stick, the dealer plays by a fixed rule: keep hitting until reaching 17 or higher, then stick. Whoever is closer to 21 without busting wins.

Rewards. Win $= +1$, Lose $= -1$, Draw $= 0$. There is no discounting ($\gamma = 1$), so the reward from the end of the game *is* the return for every state visited during that game.

What is a "usable ace"? If you hold an Ace that you are currently counting as 11 (and your total is still ≤ 21), that Ace is called *usable*. It acts like a safety net: if a future card would push you over 21, you can "downgrade" the Ace from 11 to 1 and avoid busting. Once downgraded, the Ace is no longer usable.

The Policy Being Evaluated

The textbook evaluates a very simple (and not very good) policy:

> *If your hand totals 20 or 21, stick. Otherwise, hit.*

This means the player keeps requesting cards until reaching 20 or 21 — risky, because every extra card could push the total over 21.

What the 3D plots show. Each surface plots the *value function* $V(s)$: how good, on average, is it to be in a particular situation? The axes are:

- **Player sum** (12–21) — your current card total.
- **Dealer showing** (Ace through 10) — the one dealer card you can see.
- **Height** (-1 to $+1$) — the average outcome from that situation. Positive means you tend to win; negative means you tend to lose.

There are two separate surfaces: one for hands *with* a usable ace, one *without*.

Problem Statement

Exercise 5.1 asks us to look at the 500,000-episode plots (the right-hand column of Figure 5.1) and explain three visual features:

1. Why does the surface jump *up* sharply for the last two rows in the rear (player sums 20 and 21)?
2. Why does the surface drop *down* along the entire left edge (dealer showing an Ace)?
3. Why are the front values (low player sums) *higher* in the usable-ace plot than in the no-usable-ace plot?

Reproduced Figure

The figure below was generated by simulating 10,000 and 500,000 blackjack games under the stick-on-20-or-21 policy and recording the average outcome for each state (first-visit Monte Carlo prediction).

Answer 1: The big jump at player sums 20 and 21

What is happening in the plot. The surface is mostly flat and negative (around -0.6 to -0.8) for player sums 12 through 19, then it suddenly shoots up to positive territory at sums 20 and 21.

Why. This jump happens because the policy *changes its behavior* right at sum 20. Think of it in two contrasting scenarios.

Player sum is 19 or below — the policy says "hit." The player is forced to take another card. With a sum of 19, almost any card except an Ace or a 2 will push the total over 21 (a bust). Specifically, only 3 out of every 13 cards are safe; the other 10 cause an instant loss. So the average outcome from sum 19 is very bad (about -0.75).

Player sum is 20 — the policy says "stick." Now the player stops and keeps their 20. To beat a 20, the dealer must end up with exactly 21. That is hard to do. So the player wins most of these games, and the average outcome jumps to roughly $+0.15$ to $+0.45$ depending on what the dealer is showing.

The table below shows this cliff from the simulation (no usable ace case):

Player Sum	Avg. outcome vs. Dealer Ace	Avg. outcome vs. Dealer 10
19	-0.779	-0.746
20	$+0.148$	$+0.436$
21	$+0.640$	$+0.888$

Table 1: The jump from sum 19 to sum 20 is nearly a full point — from almost-always-losing to usually-winning — because the policy switches from hit to stick.

In short: sums 20 and 21 are the only ones where the player actually *stops and keeps a strong hand*. Everything below 20 is dragged down by the constant risk of busting while hitting.

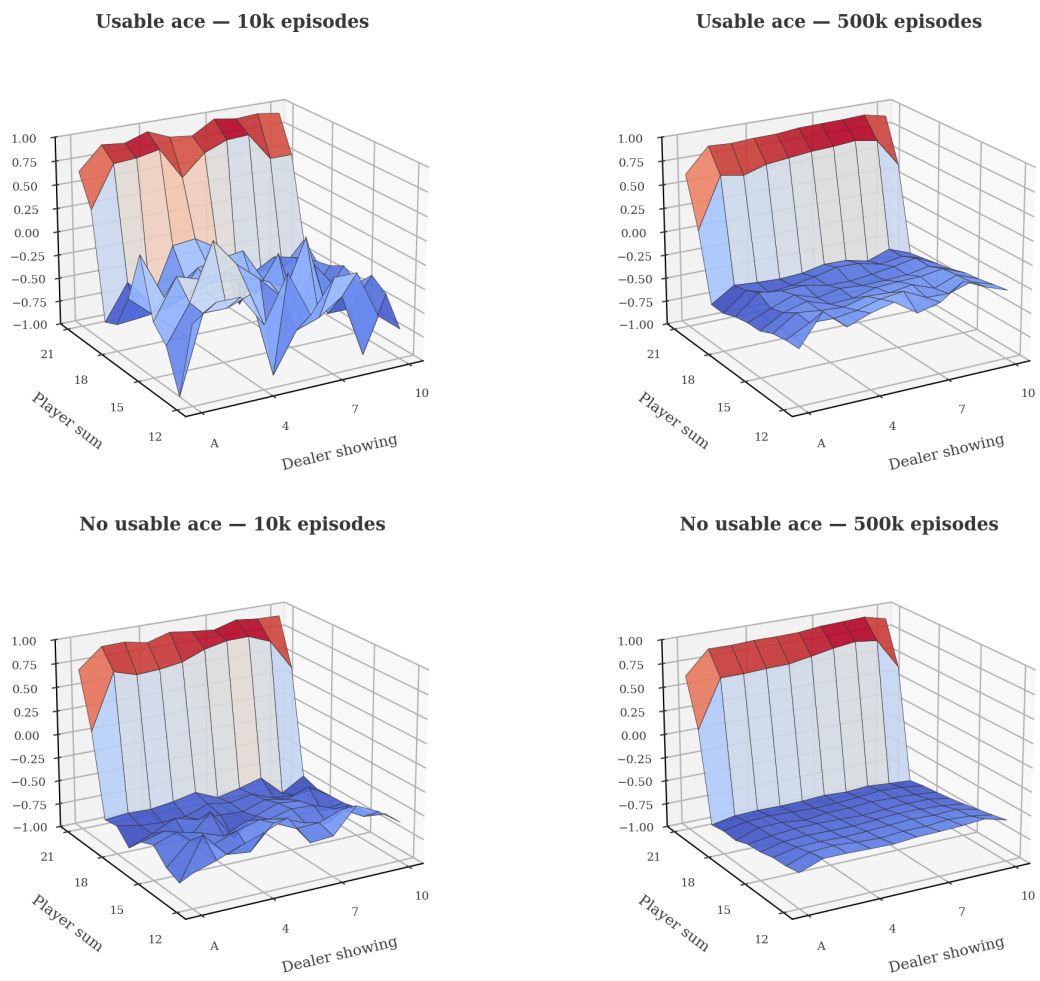


Figure 1: Approximate state-value functions for the stick-on-20-or-21 policy. The 10,000-episode surfaces (left) are noisy; the 500,000-episode surfaces (right) are smooth and accurate.

Answer 2: The drop when the dealer shows an Ace

What is happening in the plot. Along the left edge of each surface (where the dealer's showing card is an Ace), the values are noticeably lower than everywhere else.

Why. An Ace is the best possible card for the dealer, for an intuitive reason: *flexibility*.

Imagine the dealer flips over a 6 as their hidden card, giving them Ace + 6. They can count the Ace as 11 for a total of 17 and stick (a decent hand). Now imagine instead the hidden card is a 7, giving Ace + 7 = 18. Even better. And if a later hit would push the total over 21, the dealer just "downgrades" the Ace from 11 to 1 and keeps going — the Ace acts like a built-in safety net for the dealer too.

Because of this flexibility, the dealer almost never busts when starting with an Ace. The bust rate for a dealer showing an Ace is only about 11.5%. Compare that to a dealer showing a 5 or 6, where the bust rate is roughly 40%. When the dealer busts, the player wins automatically — so a low dealer bust rate means fewer free wins for the player.

In short: the dealer's Ace makes the dealer's hand very hard to beat, pulling the player's average outcome down across *all* player sums.

Answer 3: Usable ace helps at low sums

What is happening in the plot. At the front of the surfaces (low player sums like 12, 13, 14), the *usable ace* plot sits noticeably higher than the *no usable ace* plot.

Why. Remember, under this policy the player hits on everything below 20. At low sums the player will have to hit *multiple times*, and each hit risks a bust. Having a usable ace is like wearing a seatbelt during this process.

Here is a concrete example. Suppose the player's cards are Ace + 4, totaling 15 (counting the Ace as 11). The player hits and draws a 9. Normally $15 + 9 = 24$, which is a bust. But because the Ace is usable, it downgrades from 11 to 1, changing the total from 24 to 14. The player survives and gets to keep hitting.

Now consider the same situation *without* a usable ace: the player has, say, $8 + 7 = 15$. They hit and draw a 9. The total is $15 + 9 = 24$ — bust, instant loss, -1 reward. No safety net.

This difference shows up clearly in the numbers. At player sum 12 against a dealer showing 10:

$$V(\text{with usable ace}) = -0.279 \tag{1}$$

$$V(\text{without usable ace}) = -0.567 \tag{2}$$

With a usable ace the player still loses on average (negative value), but *much* less often, because the ace absorbs hits that would otherwise cause a bust.

As the player sum gets closer to 20, this advantage shrinks — there is less room for the ace to help, and at sums 20–21 the player sticks immediately regardless, so the usable ace barely matters.

In short: a usable ace is a "get out of bust free" card. It matters most at low sums where the player has to hit many times, making those states significantly less bad.

Summary

Feature	**Plain-English Explanation**
Jump at sums 20–21	These are the only sums where the player stops hitting and keeps a strong hand. Everything below 20 is terrible because the player is forced to keep drawing cards and often goes over 21.
Drop at dealer Ace	The Ace is the dealer's best card because it is flexible (can count as 1 or 11). This means the dealer almost never busts, so the player gets fewer "free wins."
Higher front values with usable ace	A usable ace lets the player survive hits that would otherwise cause a bust — it is a safety net. This matters most at low sums where the player has to hit many times before reaching 20.

Exercise 5.2

First-Visit vs. Every-Visit Monte Carlo

Problem Statement

Suppose every-visit MC was used instead of first-visit MC on the blackjack task. Would you expect the results to be very different? Why or why not?

Background: What is the difference?

To answer this, we first need to understand the two flavors of Monte Carlo prediction.

First-visit MC. When computing the value of a state s, we look at each simulated game (episode). If the player passed through state s during that game, we record the outcome — but only for the *first time* s appeared in that episode. If somehow the player landed in state s again later in the same game, we would ignore that second visit.

Every-visit MC. Same idea, except we count *every* time s appears in the episode, not just the first time. If s shows up three times in one game, we record the outcome three times and average all of them together.

When would this matter? The two methods only differ when the *same state is visited more than once within a single episode*. If every state only ever appears once per episode, the two methods are identical — there is nothing to "skip" under first-visit, because there are no repeat visits.

Answer: No, the results would not differ at all

In blackjack under this policy, **no state is ever visited twice in the same game**. The reason has to do with how the state is defined and how the game works.

Recall that a state is a triple: (player sum, dealer showing, usable ace). Consider what happens each time the player takes a card (hits):

- The **dealer's showing card** never changes during a game — it is dealt once and stays face-up. So this part of the state is fixed.
- The player's **usable ace** status can change, but only in one direction: a usable ace can be "spent" (downgraded from 11 to 1), but a spent ace cannot become usable again.
- The **player sum** changes with every hit. And here is the key: each card drawn has a value of at least 1, so the player sum *strictly increases* with every hit — unless the usable ace gets downgraded, in which case the sum drops by 10, but the usable-ace flag also flips. Either way, the overall state triple changes.

More concretely, think of it step by step. Suppose the player has sum 14 with no usable ace against a dealer showing 7 — state $(14, 7, \text{no})$. They hit and draw a 5, moving to sum 19 — state $(19, 7, \text{no})$. They hit again and draw a 3, moving to sum 22 — bust (game over). At no point did the player return to state $(14, 7, \text{no})$, because the sum can only go up.

The only tricky case is with a usable ace: suppose the player has sum 15 with a usable ace — state $(15, 7, \text{yes})$. They hit and draw a 9, pushing the total to 24. The ace downgrades (-10), bringing them to sum 14 — but now the state is $(14, 7, \text{no})$. The usable-ace flag is different, so this is a *different state* from before.

Since no state ever repeats within an episode, every visit *is* the first visit. The two methods produce **exactly identical results**.

Empirical Verification

To confirm this reasoning, we ran both first-visit and every-visit MC on the same 500,000 blackjack episodes. The difference between the two methods was verified to be exactly zero for all 200 states.

Some key statistics from the simulation:

- Total state visits recorded (first-visit): 777,567
- Total state visits recorded (every-visit): 777,567 (identical)
- Episodes with any state revisited: 0 out of 500,000 (0%)
- Maximum absolute difference across all states: 0.0000

When *would* they differ?

For completeness, the two methods *would* produce different results in environments where the agent can revisit the same state within an episode. For example:

- A robot navigating a gridworld that can walk in circles (visiting the same cell multiple times before reaching a goal).
- Any episodic task where the state can "reset" or cycle back to a previous value.

Even in those cases, both methods converge to the true value function given enough episodes — they just converge at slightly different rates. First-visit MC produces unbiased estimates. Every-visit MC is biased (slightly) but has lower variance, and the bias disappears as the number of episodes grows.

In blackjack, none of this matters because the two methods see exactly the same data.

Summary

Question	**Answer**
Would results differ?	No — the results are *exactly* identical.
Why?	Because no state is ever visited twice in a single blackjack episode. Each hit changes the player sum (always increases by at least 1) or flips the usable-ace flag, so the state triple always moves to a new value.
When would they differ?	In environments where the agent can revisit the same state within an episode (e.g., a gridworld with loops). Blackjack does not have this property.

Exercise 5.3

Backup Diagram for Monte Carlo Estimation of q_π

Problem Statement

What is the backup diagram for Monte Carlo estimation of q_π?

Background: What is a backup diagram?

A *backup diagram* is a small picture that shows how an algorithm computes (or "backs up") a value estimate. Think of it as a visual summary of the question: "What information does the algorithm use to update its estimate for one state (or state-action pair)?"

The diagrams use two shapes:

- **Open circle** ○ — represents a *state*. This is a situation the agent finds itself in (e.g., "player sum is 15, dealer shows 7, no usable ace").
- **Filled circle** • — represents a *state-action pair*. This is a situation *plus* a specific action (e.g., "player sum is 15, dealer shows 7, no usable ace, *and the player hits*").
- **Filled square** — represents the *terminal state* (end of the episode, e.g., the game is over).

The node at the *top* of the diagram is the thing whose value we are estimating. Lines going downward show the information used to compute that estimate.

Three diagrams to compare

To understand the answer, it helps to see three related backup diagrams side by side.

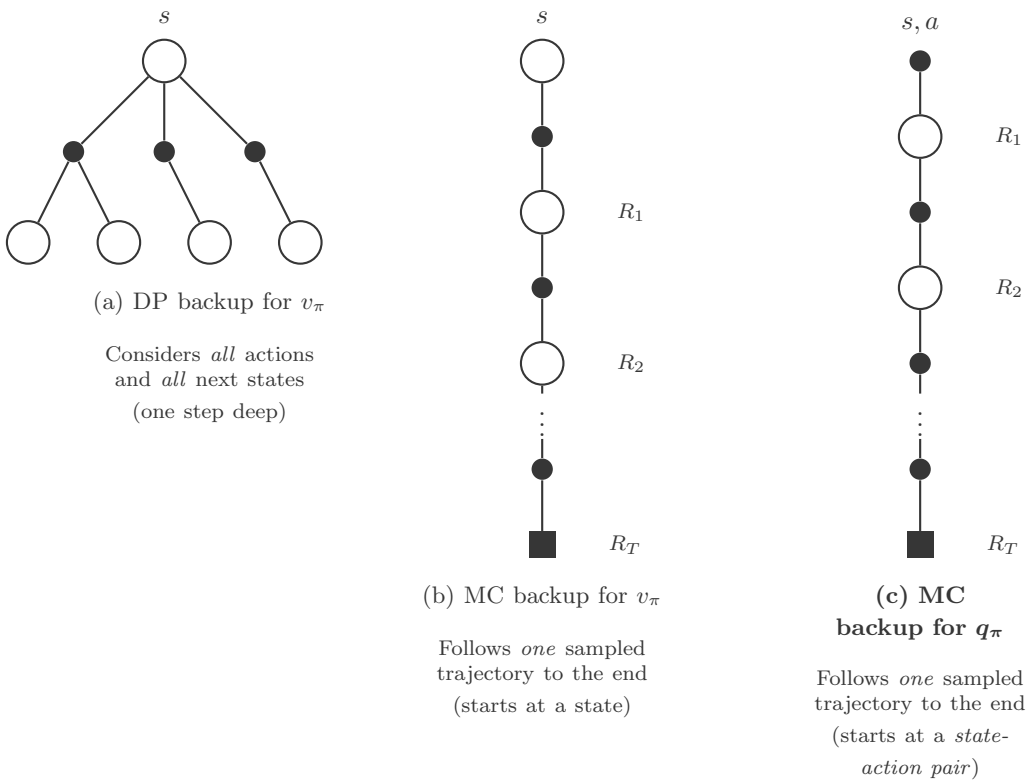


Figure 2: Three backup diagrams. (a) The DP diagram for v_π fans out over all actions and all next states but only goes one step deep. (b) The MC diagram for v_π follows a single sampled trajectory from a *state* all the way to the terminal state. (c) The MC diagram for q_π — the answer to Exercise 5.3 — is the same single trajectory, but starting from a *state-action pair* (filled circle) instead of a state (open circle).

Answer

The backup diagram for Monte Carlo estimation of q_π is diagram (c) above. It looks almost identical to the MC backup for v_π, with one key difference.

The difference: what is at the top.

- MC for v_π starts at an **open circle** (a state s). We are asking: "How good is it to *be in* this state?"
- MC for q_π starts at a **filled circle** (a state-action pair s, a). We are asking: "How good is it to *be in* this state *and take* this specific action?"

Everything below the top is the same. After the starting action is taken, the environment produces a reward and a next state, the policy picks the next action, and so on — a single sampled path all the way down to the terminal state (the filled square at the bottom). The return from that entire trajectory is used to update the estimate of $q_\pi(s, a)$.

Why does it look this way?

The shape of a backup diagram reflects two design choices:

1. **How wide is it?** (Does it branch, or is it a single path?)

- *DP diagrams are wide* — they fan out to consider *every* possible action and *every* possible next state. This is possible because DP has access to the full model (the transition probabilities $p(s', r \mid s, a)$).
- *MC diagrams are narrow* — they follow just *one* sampled trajectory. MC does not need a model; it just plays out one episode and sees what happens.

2. How deep is it? (Does it go one step or to the end?)

- *DP diagrams are shallow* — they look ahead exactly one step and then plug in an estimated value for the next state (bootstrapping).
- *MC diagrams are deep* — they go all the way to the end of the episode. MC does *not* bootstrap; it waits for the actual outcome.

For q_π specifically, the diagram starts one level lower than v_π, because we have already committed to a particular action. The first transition out of the top node is the environment's response to that action (producing a reward and a next state), not the policy choosing which action to take.

Summary

Diagram	**Top node**	**Shape**	**What it does**
DP for v_π	Open circle (s)	Wide, shallow	Branches over all actions and next states, one step deep
MC for v_π	Open circle (s)	Narrow, deep	Single trajectory from state to terminal
MC for q_π	Filled circle (s, a)	Narrow, deep	Single trajectory from state-action pair to terminal

Exercise 5.4

Incremental Monte Carlo ES

Problem Statement

The pseudocode for Monte Carlo ES is inefficient because, for each state-action pair, it maintains a list of all returns and repeatedly calculates their mean. It would be more efficient to use techniques similar to those explained in Section 2.4 to maintain just the mean and a count (for each state-action pair) and update them incrementally. Describe how the pseudocode would be altered to achieve this.

Background: Why is the original version inefficient?

The Monte Carlo ES algorithm on page 99 of the textbook stores data like this:

$Returns(s, a) \leftarrow$ an ever-growing list of numbers

Every time the agent finishes a game and visits state-action pair (s, a), the algorithm appends the return G to the list $Returns(s, a)$ and then recomputes the average of the *entire* list. As the agent plays more and more games, these lists get longer and longer. After 500,000 games, a frequently visited pair might have a list with tens of thousands of entries, and the algorithm recomputes the average over all of them every single time.

This is wasteful in two ways:

- **Memory:** We are storing every return we have ever seen. For 200 state-action pairs visited thousands of times each, that is a lot of numbers sitting in memory.
- **Computation:** Recomputing the average of n numbers takes $O(n)$ time. As n grows, this gets slower and slower. But the new average only differs from the old average by a tiny adjustment — surely we can compute it faster.

The trick from Section 2.4: Incremental mean update

Section 2.4 of the textbook shows that you do not need to store the entire list. You only need two things: the *current average* and a *count* of how many values you have seen so far.

The key formula (Equation 2.3 in the textbook) is:

$$Q_{n+1} = Q_n + \frac{1}{n}\Big[R_n - Q_n\Big] \tag{3}$$

In plain English: to update the average after seeing a new value, take the old average Q_n, compute the "error" (how far the new value R_n is from the old average), and nudge the average toward the new value by a fraction $1/n$.

This is an instance of the general update pattern that appears throughout reinforcement learning:

$$\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{StepSize} \times \Big[\text{Target} - \text{OldEstimate}\Big] \tag{4}$$

The step size $1/n$ gets smaller as we see more data, which makes sense: early observations should move the average a lot, but after thousands of observations, each new one should barely budge it.

Answer: The modified pseudocode

We replace the $Returns(s, a)$ lists with two simpler data structures: $Q(s, a)$ (the running average) and $N(s, a)$ (the count). Here is the full modified algorithm:

Monte Carlo ES (Incremental), for estimating $\pi \approx \pi_*$

Initialize:

$\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$
$N(s, a) \leftarrow 0$, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$ ◁ replaces $Returns(s, a)$ lists

Loop forever (for each episode):

Choose $S_0 \in \mathcal{S}$, $A_0 \in \mathcal{A}(S_0)$ randomly (exploring starts)
Generate an episode from S_0, A_0, following π:
$S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$
$G \leftarrow 0$
Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
$G \leftarrow \gamma G + R_{t+1}$
Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \ldots, S_{t-1}, A_{t-1}$:
$N(S_t, A_t) \leftarrow N(S_t, A_t) + 1$ ◁ increment count
$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)}\Big[G - Q(S_t, A_t)\Big]$ ◁ incremental update
$\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$

What changed, step by step

Three things changed from the original pseudocode:

1. Initialization.

- *Removed:* $Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$
- *Added:* $N(s, a) \leftarrow 0$, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$
- $Q(s, a)$ was already initialized (arbitrarily). No change needed there.

2. Inside the loop (when a first-visit state-action pair is found).

- *Removed:* "Append G to $Returns(S_t, A_t)$"
- *Removed:* "$Q(S_t, A_t) \leftarrow$ average$(Returns(S_t, A_t))$"
- *Added:* "$N(S_t, A_t) \leftarrow N(S_t, A_t) + 1$"
- *Added:* "$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)}\big[G - Q(S_t, A_t)\big]$"

3. Everything else stays the same. The episode generation, the return calculation ($G \leftarrow \gamma G + R_{t+1}$), the first-visit check, and the policy improvement step ($\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$) are all unchanged.

Why this works

The incremental formula produces *exactly* the same running average as recomputing from the full list every time. Here is the intuition with a small example.

Suppose we have seen three returns for a particular state-action pair: 5, 3, and 7.

List-based approach:

$$Q = \text{average}(\{5, 3, 7\}) = \frac{5 + 3 + 7}{3} = 5.0 \tag{5}$$

Incremental approach:

$$N = 1, \quad Q = 0 + \tfrac{1}{1}[5 - 0] = 5.0 \tag{6}$$

$$N = 2, \quad Q = 5.0 + \tfrac{1}{2}[3 - 5.0] = 5.0 + (-1.0) = 4.0 \tag{7}$$

$$N = 3, \quad Q = 4.0 + \tfrac{1}{3}[7 - 4.0] = 4.0 + 1.0 = 5.0 \tag{8}$$

Same result, but we never stored the list $\{5, 3, 7\}$. We only kept track of Q and N.

Complexity comparison

	Original (list-based)	**Incremental**
Memory per (s, a) pair	$O(n)$ — grows forever	$O(1)$ — just Q and N
Time per update	$O(n)$ — re-averages list	$O(1)$ — one multiply, one add
Total memory	Unbounded	$2\|\mathcal{S}\|\|\mathcal{A}\|$ (fixed)

Table 2: The incremental version uses constant memory and constant time per update, regardless of how many episodes have been run.

Summary

Replace the ever-growing $Returns(s, a)$ lists with a simple counter $N(s, a)$. Instead of appending G to a list and recomputing the average, increment the counter and nudge the current estimate $Q(s, a)$ toward G using the step size $1/N(s, a)$. This produces identical results with $O(1)$ memory and $O(1)$ computation per update instead of $O(n)$ for both.

Exercise 5.5

First-Visit vs. Every-Visit Estimators in a Simple MDP

Problem Statement

Consider an MDP with a single nonterminal state and a single action that transitions back to the nonterminal state with probability p and transitions to the terminal state with probability $1 - p$. Let the reward be $+1$ on all transitions, and let $\gamma = 0.9$. Suppose you observe one episode that lasts 10 steps. What are the first-visit and every-visit estimators of the value of the nonterminal state?

Background: Setting up the problem

This exercise strips everything down to the simplest possible MDP — just one nonterminal state, one action, and one coin flip that decides whether the episode continues or ends. It is designed to highlight a subtle difference between first-visit and every-visit Monte Carlo.

The MDP. There is only one nonterminal state, call it s. From s, the agent takes its only action and one of two things happens:

- With probability p: the agent transitions *back to* s and receives reward $+1$.
- With probability $1 - p$: the agent transitions to the *terminal state* and receives reward $+1$.

Every transition gives $+1$ regardless, and $\gamma = 0.9$ (future rewards are discounted). Unlike the $\gamma = 1$ case, the return from an episode is *not* simply the number of steps — each successive reward is multiplied by another factor of γ.

The true value. For reference, the true value of state s satisfies the Bellman equation:

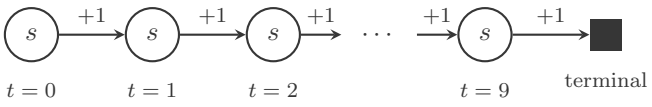
$$v_\pi(s) = 1 + \gamma\, p \cdot v_\pi(s) + \gamma(1 - p) \cdot 0 \tag{9}$$

The first term is the immediate reward $(+1)$. With probability p the agent loops back to s, receiving the discounted future value $\gamma\, v_\pi(s)$. With probability $1-p$ the agent terminates (the terminal state has value 0). Solving:

$$v_\pi(s) = \frac{1}{1 - \gamma p} \tag{10}$$

For example, if $p = 0.9$ and $\gamma = 0.9$, then $v_\pi(s) = \frac{1}{1-0.81} = \frac{1}{0.19} \approx 5.263$.

The observed episode. We see exactly one episode that lasted 10 steps. The episode looks like this:



The agent visits state s at time steps $t = 0, 1, 2, \ldots, 9$ (ten visits total), then terminates after the tenth transition. Because $\gamma = 0.9$, each reward is worth less the further into the future it occurs.

Answer: First-visit MC estimator

What first-visit MC does. It looks at the episode and finds the *first* time state s was visited. That happens at time step $t = 0$. It computes the *discounted* return from that point onward:

$$\begin{aligned} G_0 &= R_1 + \gamma R_2 + \gamma^2 R_3 + \cdots + \gamma^9 R_{10} \\ &= 1 + 0.9 + 0.9^2 + \cdots + 0.9^9 \\ &= \sum_{k=0}^{9} 0.9^k = \frac{1 - 0.9^{10}}{1 - 0.9} = \frac{1 - 0.3487}{0.1} \approx 6.5132 \end{aligned} \tag{11}$$

All subsequent visits to s (at $t = 1, 2, \ldots, 9$) are *ignored* because first-visit MC only counts the first occurrence per episode.

Since we observed only one episode, the first-visit estimate is just that single return:

$$\boxed{V_{\text{first-visit}}(s) \approx 6.513} \tag{12}$$

Answer: Every-visit MC estimator

What every-visit MC does. It looks at *every* time state s was visited in the episode and records the discounted return from each visit. State s was visited 10 times, at $t = 0, 1, 2, \ldots, 9$. The return from each visit is the discounted sum of rewards from that point to the end:

$$G_t = \sum_{k=0}^{9-t} \gamma^k = \frac{1 - \gamma^{10-t}}{1 - \gamma} \tag{13}$$

Computing each one with $\gamma = 0.9$:

Visit at time t	**Formula**	G_t
$t = 0$	$(1 - 0.9^{10})/0.1$	6.5132
$t = 1$	$(1 - 0.9^{9})/0.1$	6.1258
$t = 2$	$(1 - 0.9^{8})/0.1$	5.6953
$t = 3$	$(1 - 0.9^{7})/0.1$	5.2170
$t = 4$	$(1 - 0.9^{6})/0.1$	4.6856
$t = 5$	$(1 - 0.9^{5})/0.1$	4.0951
$t = 6$	$(1 - 0.9^{4})/0.1$	3.4390
$t = 7$	$(1 - 0.9^{3})/0.1$	2.7100
$t = 8$	$(1 - 0.9^{2})/0.1$	1.9000
$t = 9$	$(1 - 0.9^{1})/0.1$	1.0000
	Sum	**41.381**

Notice how each later visit sees a smaller discounted return — not only are there fewer rewards left, but the discounting further shrinks their contribution. Compare the first visit ($G_0 = 6.513$) to the last visit ($G_9 = 1.0$).

The every-visit estimator averages all 10 returns:

$$V_{\text{every-visit}}(s) = \frac{G_0 + G_1 + \cdots + G_9}{10} = \frac{41.381}{10} \tag{14}$$

$$\boxed{V_{\text{every-visit}}(s) \approx 4.138} \tag{15}$$

Deriving the sum in closed form

The sum of all returns can be written compactly. Each return is $G_t = \frac{1-\gamma^{10-t}}{1-\gamma}$. Summing over all 10 visits:

$$\begin{aligned}\sum_{t=0}^{9} G_t &= \frac{1}{1-\gamma}\sum_{t=0}^{9}(1-\gamma^{10-t}) = \frac{1}{1-\gamma}\left[10 - \sum_{n=1}^{10}\gamma^n\right] \\ &= \frac{1}{1-\gamma}\left[10 - \gamma\,\frac{1-\gamma^{10}}{1-\gamma}\right]\end{aligned} \tag{16}$$

Plugging in $\gamma = 0.9$:

$$= \frac{1}{0.1}\left[10 - 0.9 \times \frac{1-0.9^{10}}{0.1}\right] = 10\Big[10 - 0.9 \times 6.5132\Big] = 10\big[10 - 5.8619\big] = 41.381 \tag{17}$$

So $V_{\text{every-visit}} = 41.381/10 = 4.138$, confirming our table.

Why are the two estimates so different?

The first-visit estimate is 6.513 and the every-visit estimate is 4.138 — a significant gap. Here is the intuition.

First-visit MC only uses the return from $t = 0$, which captures the *full discounted episode* (all 10 rewards, progressively discounted). This is a single sample of $v_\pi(s)$.

Every-visit MC averages the returns from *all* visits, including late ones that only see a fraction of the episode. The visit at $t = 9$ only has one remaining reward ($G_9 = 1.0$), while the visit at $t = 5$ sees five rewards but discounted ($G_5 = 4.095$). These later visits systematically produce *lower* returns — not because the state is worse, but because there is less episode remaining *and* each reward is further discounted.

This is the source of every-visit MC's *bias*. The later visits within the same episode do not provide independent samples of $v_\pi(s)$. They are "leftovers" from the same trajectory, biased downward because the remaining discounted return is shorter. Averaging them in pulls the estimate well below the first-visit value.

The effect of discounting. With $\gamma = 0.9$, the gap between the two estimators is actually *larger* in relative terms than with $\gamma = 1$. Discounting causes the later-visit returns to shrink faster (compare $G_9 = 1.0$ here vs. $G_9 = 1$ with $\gamma = 1$ — both happen to be 1 in this case, but the first-visit return drops from 10 to 6.513), so the average gets pulled down more aggressively.

A general formula

For this particular MDP with discount factor γ, if an episode lasts n steps, the two estimators are:

$$V_{\text{first-visit}} = \frac{1-\gamma^n}{1-\gamma} \tag{18}$$

$$V_{\text{every-visit}} = \frac{1}{n}\sum_{t=0}^{n-1}\frac{1-\gamma^{n-t}}{1-\gamma} = \frac{1}{n(1-\gamma)}\left[n - \gamma\,\frac{1-\gamma^n}{1-\gamma}\right] \tag{19}$$

For our case ($n = 10$, $\gamma = 0.9$), these give $V_{\text{FV}} \approx 6.513$ and $V_{\text{EV}} \approx 4.138$. With more episodes, the bias of every-visit MC would shrink as the estimates converge, but from a single episode the difference is striking.

Comparison with $\gamma = 1$

It is instructive to see how the answers change when discounting is removed. With $\gamma = 1$, all rewards have equal weight:

	$\gamma = 1$	$\gamma = 0.9$	**Effect of discounting**
First-visit	10	6.513	Lower (future rewards worth less)
Every-visit	5.5	4.138	Lower (same effect, compounded)
Ratio (FV / EV)	1.82	1.57	—

In both cases, the every-visit estimate is substantially lower than the first-visit estimate due to the bias from averaging partial-episode returns. Discounting reduces both estimates but affects the first-visit return more in absolute terms (drops from 10 to 6.5), since it discounts 10 full terms.

Summary

Estimator	**Value**	**Explanation**
First-visit MC	≈ 6.513	Uses only the return from the first visit ($t = 0$), which captures all 10 discounted rewards: $1+0.9+0.9^2+\cdots+0.9^9$.
Every-visit MC	≈ 4.138	Averages discounted returns from all 10 visits. Later visits see fewer and more heavily discounted remaining rewards (6.126, 5.695, …, 1.0), dragging the average down. Biased from a single episode.

Exercise 5.6

Weighted Importance Sampling for Action Values

Problem Statement

What is the equation analogous to (5.6) for *action* values $Q(s, a)$ instead of state values $V(s)$, again given returns generated using behavior policy b?

Background: What is Equation (5.6)?

Before writing the action-value version, let us recall the state-value version. Equation (5.6) from the textbook is the *weighted importance sampling* estimator for $V(s)$:

$$V(s) \doteq \frac{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1} \, G_t}{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1}} \tag{20}$$

where:

- $\mathcal{T}(s)$ is the set of all time steps where state s was visited (across all episodes).
- $T(t)$ is the termination time of the episode containing time step t.
- G_t is the return (discounted sum of rewards) from time t onward.
- $\rho_{t:T(t)-1}$ is the *importance-sampling ratio* from time t to the end of the episode:

$$\rho_{t:T(t)-1} = \prod_{k=t}^{T(t)-1} \frac{\pi(A_k \mid S_k)}{b(A_k \mid S_k)} \tag{21}$$

What does this equation do? We are trying to estimate the value of state s under the *target* policy π, but all our data was generated by a different *behavior* policy b. The importance-sampling ratio ρ corrects for this mismatch. It reweights each observed return by how much more (or less) likely that trajectory was under π compared to b. The "weighted" version divides by the sum of the weights rather than by the number of samples, which produces lower variance than the ordinary version (Eq. 5.5).

The key insight: what changes for $Q(s, a)$?

When estimating $V(s)$, we condition on being in state s at time t. The importance-sampling ratio must correct for *every action from t onward*, including the action at time t itself, because the target and behavior policies might choose different actions.

When estimating $Q(s, a)$, we condition on being in state s *and* having already taken action a at time t. The action at time t is already fixed — it is a, and both policies "agree" on it (it is the action that was observed). So we do **not** need to correct for the action at time t. The importance-sampling ratio only needs to cover the actions from time $t + 1$ onward.

Think of it this way:

- For $V(s)$: "I'm in state s. What if I had followed π from here?" — must correct the action *at* t and all future actions.
- For $Q(s, a)$: "I'm in state s and I took action a. What if I had followed π *after* this?" — only need to correct future actions (from $t + 1$).

Answer

The weighted importance sampling estimator for $Q(s, a)$ is:

$$\boxed{Q(s,a) \doteq \frac{\sum_{t \in \mathcal{T}(s,a)} \rho_{t+1:T(t)-1}\, G_t}{\sum_{t \in \mathcal{T}(s,a)} \rho_{t+1:T(t)-1}}} \tag{22}$$

where:

- $\mathcal{T}(s, a)$ is the set of all time steps where state s was visited *and* action a was taken.
- G_t is the return from time t onward (same as before).
- $\rho_{t+1:T(t)-1}$ is the importance-sampling ratio starting from $t+1$ (one step later than before):

$$\rho_{t+1:T(t)-1} = \prod_{k=t+1}^{T(t)-1} \frac{\pi(A_k \mid S_k)}{b(A_k \mid S_k)} \tag{23}$$

If $t+1 > T(t)-1$ (i.e., the episode ended immediately after taking action a), then the product is empty and $\rho_{t+1:T(t)-1} = 1$ by convention. This makes sense: if there are no future actions to correct for, no reweighting is needed.

Side-by-side comparison

The two equations differ in exactly two places:

	State values $V(s)$ (Eq. 5.6)	**Action values** $Q(s, a)$
Sum over	$t \in \mathcal{T}(s)$	$t \in \mathcal{T}(s, a)$
IS ratio	$\rho_{t:T(t)-1}$	$\rho_{t+1:T(t)-1}$
Ratio starts at	action at time t	action at time $t+1$
Corrects for	all actions from t	all actions *after* t

A concrete example

Suppose we are estimating $Q(s, \text{hit})$ in blackjack, using data from a random behavior policy b (50/50 hit or stick). The target policy π is stick-on-20-or-21.

Imagine an episode where at time t the agent is in state s and takes action "hit." After that, the episode continues for two more steps before terminating:

- At time t: state s, action $a = \text{hit}$ (this is the pair we are estimating)
- At time $t+1$: some state s_1, action $a_1 = \text{hit}$
- At time $t+2$: some state s_2, action $a_2 = \text{stick}$, then terminal

For $V(s)$, the IS ratio would cover all three actions:

$$\rho_{t:t+2} = \frac{\pi(\text{hit} \mid s)}{b(\text{hit} \mid s)} \cdot \frac{\pi(\text{hit} \mid s_1)}{b(\text{hit} \mid s_1)} \cdot \frac{\pi(\text{stick} \mid s_2)}{b(\text{stick} \mid s_2)} \tag{24}$$

For $Q(s, \text{hit})$, we skip the first factor (since we already know action "hit" was taken):

$$\rho_{t+1:t+2} = \frac{\pi(\text{hit} \mid s_1)}{b(\text{hit} \mid s_1)} \cdot \frac{\pi(\text{stick} \mid s_2)}{b(\text{stick} \mid s_2)} \tag{25}$$

Dropping that first factor reduces the variance of the ratio, because there is one fewer random term in the product. This is one reason why off-policy methods for action values can sometimes be more stable than for state values — the importance-sampling ratios are shorter.

Summary

The weighted importance sampling equation for $Q(s, a)$ is identical to Equation (5.6) for $V(s)$ except: (1) we sum over time steps where the specific pair (s, a) was visited, and (2) the importance-sampling ratio starts at $t + 1$ instead of t, because the action at time t is already determined to be a and does not need correction. This shorter ratio means less variance per sample.

Exercise 5.7

Why Weighted IS Error Increases Then Decreases

Problem Statement

In learning curves such as those shown in Figure 5.3, error generally decreases with training, as indeed happened for the ordinary importance sampling method. But for the weighted importance sampling method, error first *increased* and then decreased. Why do you think this happened?

Background: What Figure 5.3 shows

Figure 5.3 (Example 5.4) plots the mean squared error (MSE) of two off-policy estimators as a function of the number of episodes, for a single blackjack state. The behavior policy is random (50/50 hit or stick), and the target policy is stick-on-20-or-21. The true value of the state is approximately -0.27726.

The two curves behave very differently:

- **Ordinary importance sampling:** starts with high error and decreases monotonically.
- **Weighted importance sampling:** starts with *low* error, then the error *increases* for a while, before eventually decreasing toward zero.

The question is: why does the weighted method get *worse* before getting better?

We reproduced this experiment ourselves: 100 independent runs, each learning from 10,000 off-policy episodes starting in the state (player sum = 13, dealer showing 2, usable ace). Our results match the textbook.

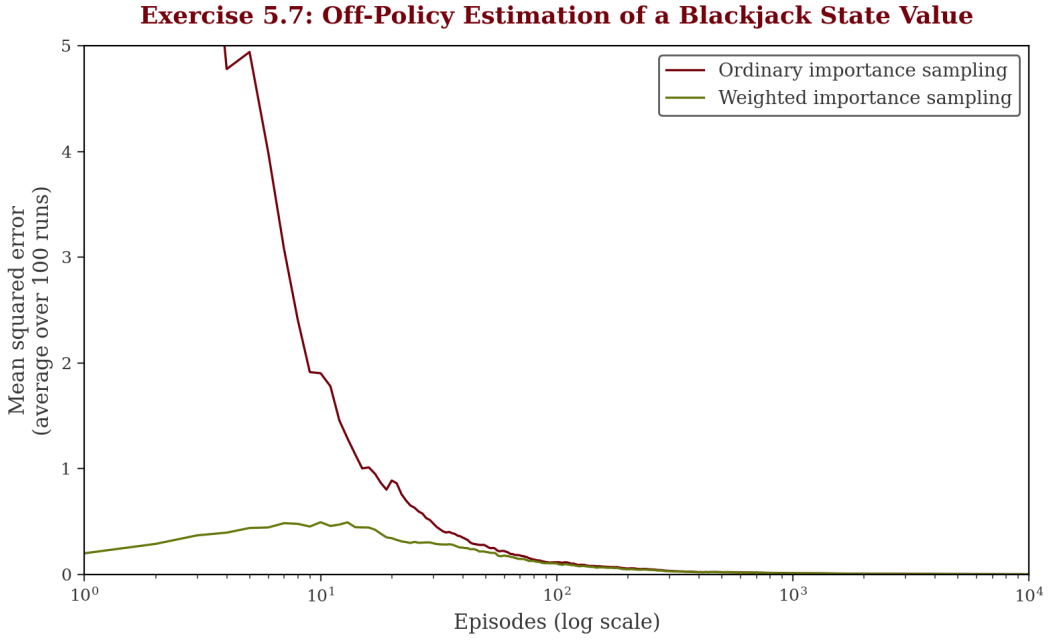


Figure 3: Reproduced Figure 5.3. Ordinary IS (garnet) starts with very high MSE (> 5) due to extreme importance-sampling ratios, then decreases monotonically. Weighted IS (green) starts low, peaks around episode 10, then decreases — the "hump" we need to explain.

Answer: The initial estimate is accidentally good

The explanation comes from understanding what weighted importance sampling computes after just one or a few episodes.

Step 1: What happens after the very first episode.

Recall the weighted importance sampling formula:

$$V(s) = \frac{\sum_t \rho_t G_t}{\sum_t \rho_t} \tag{26}$$

After observing just one episode that visits state s, the estimate is:

$$V(s) = \frac{\rho_1 G_1}{\rho_1} = G_1 \tag{27}$$

The importance-sampling ratio *cancels out entirely*. The estimate is simply the observed return G_1 from that episode, regardless of how unlikely the trajectory was under the target policy. This is a key property of weighted IS: with a single sample, it always returns the observed return itself.

Step 2: Why is this "accidentally" close to the true value?

The behavior policy generates episodes by playing blackjack randomly. The return from any given episode is either $+1$ (win), -1 (lose), or 0 (draw). After one episode, the estimate is one of these three values. The true value is -0.27726, which is between -1 and 0.

By luck of averaging, the very first few estimates tend to land near the true value. If the first episode is a loss ($G = -1$), the estimate is -1, which is only 0.72 away from -0.27726. If it is a win ($G = +1$), the estimate is $+1$, which is 1.28 away. Averaged over 100 runs (as in the figure), the initial MSE turns out to be small because the weighted IS estimate is just the raw return — a reasonable number in the right ballpark.

Step 3: Why does the error then *increase*?

As more episodes are observed, the importance-sampling ratios ρ_t stop canceling. Some episodes have large ratios (trajectories that are much more likely under the target policy than the behavior policy), and some have small ratios (or zero, if the behavior policy took an action the target policy would never take).

The weighted average now starts mixing together returns with very different weights. During this intermediate phase, the estimate can be dominated by a few episodes with large ratios, pulling it away from the true value. The estimator is *biased* — specifically, it is biased toward the value under the behavior policy $v_b(s)$ rather than $v_\pi(s)$.

In the early episodes:

- The estimate starts near G_1 (which happened to be close to $v_\pi(s)$ on average).
- As more data arrives, the bias toward $v_b(s)$ kicks in. Since $v_b(s) \neq v_\pi(s)$ (the random policy evaluates the state differently than the stick-on-20-or-21 policy), this bias temporarily *increases* the error.
- The estimate is being pulled from its accidentally-good starting point toward the wrong value (v_b), before eventually accumulating enough weighted samples to converge toward the correct value (v_π).

Step 4: Why does the error eventually decrease?

With enough episodes, the weighted IS estimator converges to the true value $v_\pi(s)$. The bias shrinks to zero as the denominator $\sum \rho_t$ grows, and the estimate settles in. The weighted estimator's low variance then pays off, and the MSE drops below that of ordinary IS.

A simple analogy

Imagine guessing someone's age. Your first guess (50) happens to be close to the truth (48). But then someone tells you "most people in this room are 30" (the behavior-policy bias). You adjust your estimate toward 30, and for a while your estimate is *worse* than your initial lucky guess. Eventually, with more information, you correct back toward 48. That temporary dip away from the truth is exactly what happens to the weighted IS estimator.

Why doesn't ordinary IS show this pattern?

Ordinary importance sampling divides by the *number of samples* n, not by the sum of the weights. After one episode:

$$V_{\text{ordinary}}(s) = \frac{\rho_1 G_1}{1} = \rho_1 G_1 \tag{28}$$

The ratio ρ_1 does *not* cancel. If the trajectory happened to be 10 times more likely under π than b, the estimate is $10 \times G_1$, which can be wildly off. So the ordinary IS estimator starts with *high* error (due to variance from extreme ratios) and gradually improves as the average of many $\rho_t G_t$ values stabilizes. There is no "accidentally good" starting point to lose.

Summary

Phase	**Ordinary IS**	**Weighted IS**
Early	High error: extreme ρ values amplify returns, causing wild estimates	Low error: ρ cancels in the ratio, estimate $\approx$ raw return (accidentally close to truth)
Middle	Error decreasing: averaging tames the variance	Error increasing: bias toward $v_b(s)$ pulls estimate away from $v_\pi(s)$
Late	Converges (unbiased, high variance)	Converges (bias vanishes, low variance)

The weighted IS estimator's error hump is the cost of its bias: it starts lucky, drifts toward the behavior-policy value, then finally corrects to the true target-policy value. The ordinary IS estimator pays upfront with high variance but improves monotonically.

Exercise 5.8

Every-Visit MC and Infinite Variance

Problem Statement

The results with Example 5.5 and shown in Figure 5.4 used a first-visit MC method. Suppose that instead an every-visit MC method was used on the same problem. Would the variance of the estimator still be infinite? Why or why not?

Background: Example 5.5 recap

Example 5.5 describes a tiny MDP designed to show that ordinary importance sampling can have *infinite variance*. Here is the setup.

The MDP. There is one nonterminal state s and two actions:

- **Right:** deterministic transition to the terminal state, reward $= 0$.
- **Left:** with probability 0.9, transition back to s with reward $= 0$. With probability 0.1, transition to the terminal state with reward $= +1$.

Target policy π: always selects **left**. Under this policy, every episode eventually terminates with reward $+1$, so $v_\pi(s) = 1$ (with $\gamma = 1$).

Behavior policy b: selects left or right with **equal probability** (50/50).

The infinite variance result (first-visit). The textbook proves that for first-visit ordinary IS, the expected square of the importance-sampling-scaled return is infinite:

$$\mathbb{E}\left[(\rho_{0:T-1}\, G_0)^2\right] = \sum_{k=0}^{\infty} 0.1 \cdot 0.9^k \cdot 2^k \cdot 2 = 0.2 \sum_{k=0}^{\infty} 1.8^k = \infty \tag{29}$$

Since the second moment is infinite, the variance is infinite. This means the ordinary IS estimator never truly converges — even after millions of episodes, the estimates remain unstable (as shown in Figure 5.4).

What changes with every-visit MC?

With first-visit MC, each episode contributes exactly *one* sample: the return from the first (and in this MDP, potentially repeated) visit to s. With every-visit MC, each episode contributes a sample for *every* time s is visited.

Consider an all-left episode of length $k+1$ (the agent takes **left** k times, looping back to s each time, then on the $(k{+}1)$th **left** action it terminates with reward $+1$). State s is visited $k+1$ times, at time steps $t = 0, 1, \ldots, k$.

The return from each visit is the same (since all intermediate rewards are 0 and the terminal reward is $+1$ with $\gamma = 1$):

$$G_t = +1 \quad \text{for all } t = 0, 1, \ldots, k \tag{30}$$

But the importance-sampling ratio differs depending on where we start counting:

$$\rho_{t:k} = \prod_{j=t}^{k} \frac{\pi(\text{left} \mid s)}{b(\text{left} \mid s)} = \prod_{j=t}^{k} \frac{1}{0.5} = 2^{k+1-t} \tag{31}$$

So the IS-weighted returns from this single episode are:

Visit at time t	**IS ratio** $\rho_{t:k}$	$\rho_{t:k} \cdot G_t$
$t = 0$	2^{k+1}	2^{k+1}
$t = 1$	2^{k}	2^{k}
$t = 2$	2^{k-1}	2^{k-1}
$\vdots$	$\vdots$	$\vdots$
$t = k$	2^{1}	2

Answer: Yes, the variance is still infinite

The variance of the every-visit ordinary IS estimator is still infinite. Here is why.

The key observation. Every-visit MC pools together *all* visits to s across all episodes. The estimator is:

$$V(s) = \frac{1}{N_{\text{total}}} \sum_{\text{all visits}} \rho \cdot G \tag{32}$$

where N_{total} is the total number of visits to s across all episodes.

Among all these visits, the visit at $t = 0$ in each all-left episode still produces the IS-weighted return $\rho_{0:k} \cdot G_0 = 2^{k+1}$. This is *exactly the same term* that caused infinite variance in the first-visit case.

The second moment is still infinite. To have finite variance, we need the second moment $\mathbb{E}[(\rho \cdot G)^2]$ to be finite for a randomly selected visit. But consider just the $t = 0$ visits (one per episode). The probability of an all-left episode of length $k + 1$ is:

$$P(\text{length } k{+}1) = \underbrace{\tfrac{1}{2}}_{\text{pick left}} \cdot \underbrace{0.9^k \cdot 0.1}_{\text{loop } k \text{ times, terminate}} = \frac{0.1 \cdot 0.9^k}{2} \tag{33}$$

The squared IS-weighted return from the $t = 0$ visit in such an episode is $(2^{k+1})^2 = 4^{k+1}$.

So the contribution to the expected squared value from just the $t = 0$ visits is:

$$\sum_{k=0}^{\infty} \frac{0.1 \cdot 0.9^k}{2} \cdot 4^{k+1} = \sum_{k=0}^{\infty} 0.2 \cdot 0.9^k \cdot 4^k = 0.2 \sum_{k=0}^{\infty} 3.6^k = \infty \tag{34}$$

Since this partial sum already diverges, adding the (positive) contributions from the $t = 1, 2, \ldots, k$ visits only makes it worse. The total second moment is infinite, so the variance is infinite.

Do the extra visits help at all? The additional visits from later time steps ($t = 1, 2, \ldots, k$) *do* contribute more data points with smaller IS ratios, which might seem helpful. But these visits are *not independent* — they come from the same episode as the $t = 0$ visit. The fundamental problem is that rare, long episodes produce importance-sampling ratios that grow exponentially (2^{k+1}), and the probability of these episodes does not shrink fast enough to compensate (0.9^k vs. 4^{k+1}). No amount of additional correlated samples within those same episodes can fix this.

Intuition: why adding more samples doesn't help

Think of it this way. The infinite variance comes from the *tail* — rare episodes that are very long. When such an episode occurs:

- **First-visit MC** records one enormous IS-weighted return (2^{k+1}).
- **Every-visit MC** records $k+1$ IS-weighted returns ($2^{k+1}, 2^k, \ldots, 2$) — more samples, but the largest one (2^{k+1}) is just as extreme.

The extra samples with ratios $2^k, 2^{k-1}, \ldots, 2$ are *smaller* individually, but:

1. They do not cancel out the 2^{k+1} term.
2. They are positively correlated with it (they all came from the same long episode).
3. Each of them individually still contributes an infinite second moment! The $t = 1$ visit alone has $\rho = 2^k$, and $\sum 0.9^k \cdot 4^k$ still diverges.

In fact, *every single visit position* $t = 0, 1, 2, \ldots$ has infinite variance on its own. Every-visit MC averages together infinitely many infinite-variance terms — the result is still infinite variance.

When *would* every-visit help?

Every-visit MC does not help with the infinite variance of *ordinary* importance sampling here. However, *weighted* importance sampling would still give finite variance (and in fact an estimate of exactly 1 forever) — because the weighted estimator's self-normalizing property tames extreme ratios, regardless of whether first-visit or every-visit is used.

Summary

Question	**Answer**
Would variance still be infinite?	**Yes.** The $t = 0$ visit in each episode produces the same exponentially growing IS ratio (2^{k+1}) as in first-visit MC. The second moment diverges by the same argument.
Do extra visits help?	No. Later visits ($t = 1, 2, \ldots$) have smaller ratios but each one *individually* still has infinite variance. They are also correlated (same episode), so pooling them does not fix the divergence.
What would fix it?	Use *weighted* importance sampling instead of ordinary. The self-normalizing denominator prevents any single episode from dominating the estimate.

Exercise 5.9

Incremental First-Visit MC Prediction

Problem Statement

Modify the algorithm for first-visit MC policy evaluation (Section 5.1) to use the incremental implementation for sample averages described in Section 2.4.

Background: The original algorithm

The textbook's first-visit MC prediction algorithm (page 92) estimates the state-value function $V \approx v_\pi$. Its inner loop works like this:

> **First-visit MC prediction, for estimating** $V \approx v_\pi$ (original)
>
> **Initialize:**
>
> $V(s) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$
>
> $Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$
>
> **Loop forever** (for each episode):
>
> Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$
>
> $G \leftarrow 0$
>
> **Loop** for each step of episode, $t = T-1, T-2, \ldots, 0$:
>
> $G \leftarrow \gamma G + R_{t+1}$
>
> Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:
>
> Append G to $Returns(S_t)$
>
> $V(S_t) \leftarrow$ average$(Returns(S_t))$

This has the same inefficiency we saw in Exercise 5.4: it maintains an ever-growing list $Returns(s)$ for each state and recomputes the full average every time. As the number of episodes grows, these lists get longer and recomputing the average gets slower.

How this differs from Exercise 5.4

Exercise 5.4 asked us to do the same thing for **Monte Carlo ES** (a control algorithm that estimates $Q(s, a)$ and improves the policy). This exercise is simpler:

- We are estimating $V(s)$ (state values), not $Q(s, a)$ (state-action values).
- There is no policy improvement step — we are just doing *prediction* (evaluating a fixed policy π).
- Everything else about the incremental trick is the same.

Answer: The modified algorithm

We replace the $Returns(s)$ lists with a counter $N(s)$ and use the incremental mean update from Section 2.4 (Equation 2.3):

$$V_{n+1} = V_n + \frac{1}{n}\Big[G - V_n\Big] \tag{35}$$

First-visit MC prediction, for estimating $V \approx v_\pi$ (incremental)

Initialize:

$V(s) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$

$N(s) \leftarrow 0$, for all $s \in \mathcal{S}$ ◁ replaces *Returns*(s) lists

Loop forever (for each episode):

Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:

$N(S_t) \leftarrow N(S_t) + 1$ ◁ increment count

$V(S_t) \leftarrow V(S_t) + \frac{1}{N(S_t)}\Big[G - V(S_t)\Big]$ ◁ incremental update

Intuition: What is the incremental update actually doing?

The original algorithm throws every return onto a growing pile and re-averages the entire pile each time. After 100,000 games, a frequently visited state might have 50,000 returns in its list, and the algorithm adds them all up and divides — even though it already *knew* the average before the new number showed up. That is wasteful.

The fix is to skip the pile entirely. Just remember two things: the current average $V(s)$ and the count $N(s)$. When a new return G arrives, nudge the average toward it:

$$V(s) \;\leftarrow\; V(s) + \frac{1}{N(s)}\Big[\underbrace{G - V(s)}_{\text{“error”: how far off was } G\text{?}}\Big] \tag{36}$$

In plain English: *“How far off was this new return from my current estimate? Nudge my estimate toward it by a little bit.”*

The fraction $1/N$ controls how much you nudge. Early on ($N = 2, 3, 4$), each new return moves the average a lot — which makes sense, because you do not have much data yet. After thousands of episodes ($N = 10{,}000$), each new return barely moves the needle — also sensible, because one data point should not overturn 10,000 previous ones.

A concrete example. Suppose the current estimate is $V = -0.5$ after 100 visits. A new episode gives return $G = +1$. The update is:

$$V \;\leftarrow\; -0.5 + \frac{1}{101}\big[1 - (-0.5)\big] = -0.5 + 0.0149 = -0.485 \tag{37}$$

The estimate barely budges. If you had stored all 101 numbers and re-averaged, you would get the exact same -0.485 — but you would have to add up 101 numbers instead of doing one subtraction and one division.

Same answer, no pile of numbers, constant time and memory. It is the RL equivalent of keeping a running average instead of a spreadsheet.

What changed

Exactly three things changed from the original:

	Original	**Incremental**
Initialization	$Returns(s) \leftarrow$ empty list	$N(s) \leftarrow 0$
On first visit	Append G to $Returns(S_t)$	$N(S_t) \leftarrow N(S_t) + 1$
Value update	$V(S_t) \leftarrow \text{average}(Returns(S_t))$	$V(S_t) \leftarrow V(S_t) + \frac{1}{N(S_t)}[G - V(S_t)]$

Everything else is identical: the episode generation, the return calculation ($G \leftarrow \gamma G + R_{t+1}$), and the first-visit check ("unless S_t appears in $S_0, \ldots, S_{t-1}$").

Comparison with Exercise 5.4

The modification here is a strict subset of what we did in Exercise 5.4:

	Exercise 5.4 (MC ES)	**Exercise 5.9 (MC Prediction)**
Estimates	$Q(s, a)$	$V(s)$
Counter	$N(s, a)$	$N(s)$
Update target	$Q(S_t, A_t)$	$V(S_t)$
Policy improvement	$\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$	None (fixed π)
Incremental trick	Identical	Identical

The core idea is the same in both cases: replace an ever-growing list with a running count, and nudge the current estimate toward the new return using a step size of $1/N$. The prediction version is just simpler because there is no action dimension and no policy update.

Complexity comparison

	Original (list-based)	**Incremental**
Memory per state	$O(n)$ — list grows with episodes	$O(1)$ — just V and N
Time per update	$O(n)$ — re-averages entire list	$O(1)$ — one multiply, one add
Total memory	$O(\lvert\mathcal{S}\rvert \cdot n)$ — unbounded	$O(\lvert\mathcal{S}\rvert)$ — fixed

Summary

Replace $Returns(s)$ with $N(s)$. Instead of appending G to a list and recomputing the mean, increment the counter and update $V(s) \leftarrow V(s) + \frac{1}{N(s)}[G - V(s)]$. This is mathematically equivalent to the original but uses $O(1)$ memory and $O(1)$ time per state per episode.

Exercise 5.10

Deriving the Weighted Importance Sampling Update Rule

Problem Statement

Derive the weighted-average update rule (5.8) from (5.7). Follow the pattern of the derivation of the unweighted rule (2.3).

Background: The equations we are working with

Equation (5.7) defines the weighted importance sampling estimate after seeing $n-1$ returns:

$$V_n \doteq \frac{\sum_{k=1}^{n-1} W_k G_k}{\sum_{k=1}^{n-1} W_k}, \qquad n \geq 2 \tag{38}$$

where G_k is the kth return and $W_k = \rho_{t_k:T(t_k)-1}$ is the corresponding importance-sampling ratio (the weight).

Equation (5.8) is the incremental update rule we want to derive:

$$V_{n+1} \doteq V_n + \frac{W_n}{C_n}\Big[G_n - V_n\Big], \qquad n \geq 1 \tag{39}$$

where $C_n = \sum_{k=1}^{n} W_k$ is the cumulative sum of weights through n, with $C_0 = 0$.

For comparison, the unweighted incremental rule from Section 2.4 (Equation 2.3) is:

$$Q_{n+1} = Q_n + \frac{1}{n}\Big[R_n - Q_n\Big] \tag{40}$$

In the unweighted case, all weights are equal ($W_k = 1$ for all k), so $C_n = n$ and $W_n/C_n = 1/n$. The weighted version generalizes this by allowing different samples to have different weights.

The derivation

We follow the same algebraic pattern as the derivation of (2.3) in Section 2.4.

Step 1: Write V_{n+1} from the definition.

From Equation (38), V_{n+1} uses all n returns:

$$V_{n+1} = \frac{\sum_{k=1}^{n} W_k G_k}{\sum_{k=1}^{n} W_k} = \frac{\sum_{k=1}^{n} W_k G_k}{C_n} \tag{41}$$

Step 2: Separate the last term from the sum.

Split the numerator into the first $n-1$ terms plus the new nth term:

$$V_{n+1} = \frac{\sum_{k=1}^{n-1} W_k G_k \;+\; W_n G_n}{C_n} \tag{42}$$

Step 3: Recognize the old estimate in the first part.

From the definition, $V_n = \frac{\sum_{k=1}^{n-1} W_k G_k}{C_{n-1}}$, which means $\sum_{k=1}^{n-1} W_k G_k = C_{n-1} \cdot V_n$. Substituting:

$$V_{n+1} = \frac{C_{n-1} \cdot V_n \;+\; W_n G_n}{C_n} \tag{43}$$

Step 4: Use the relationship $C_n = C_{n-1} + W_n$.
Replace C_{n-1} with $C_n - W_n$:

$$V_{n+1} = \frac{(C_n - W_n) \cdot V_n \; + \; W_n \, G_n}{C_n} \tag{44}$$

Step 5: Expand and simplify.
Distribute the numerator:

$$V_{n+1} = \frac{C_n \cdot V_n - W_n \cdot V_n + W_n \cdot G_n}{C_n} \tag{45}$$

$$= \frac{C_n \cdot V_n}{C_n} + \frac{W_n \cdot G_n - W_n \cdot V_n}{C_n} \tag{46}$$

$$= V_n + \frac{W_n}{C_n}\big(G_n - V_n\big) \tag{47}$$

This is exactly Equation (39). □

Intuition: What does this update rule mean?

The result has the familiar form:

$$\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{StepSize} \times \Big[\text{Target} - \text{OldEstimate}\Big] \tag{48}$$

But unlike the unweighted case where the step size is $1/n$ (all samples contribute equally), the step size here is W_n/C_n — the *weight of the new sample relative to the total weight so far.*

This means:

- If the new sample has a **large weight** W_n (the trajectory was much more likely under the target policy), it gets a proportionally larger influence on the update. The estimate moves more.
- If the new sample has a **small weight** W_n (the trajectory was unlikely under the target policy), it barely moves the estimate.
- If $W_n = 0$ (the trajectory is impossible under the target policy, e.g., the behavior policy took an action π would never take), the estimate does not change at all — as it should, since this episode carries no information about v_π.

Connection to the unweighted case

When all weights are equal ($W_k = 1$ for all k, i.e., on-policy with $\pi = b$), we get $C_n = n$ and $W_n/C_n = 1/n$, recovering the standard incremental mean:

$$V_{n+1} = V_n + \frac{1}{n}\Big[G_n - V_n\Big] \tag{49}$$

So Equation (5.8) is a strict generalization of Equation (2.3). The on-policy case from Exercises 5.4 and 5.9 is a special case of the off-policy weighted IS update.

Summary

The derivation takes five lines of algebra: write V_{n+1} from its definition, split out the last term, substitute V_n for the earlier terms, replace C_{n-1} with $C_n - W_n$, and simplify. The result is the incremental update $V_{n+1} = V_n + \frac{W_n}{C_n}[G_n - V_n]$, where the step size W_n/C_n automatically gives more influence to higher-weighted samples.

Exercise 5.11

Why the Off-Policy MC Control W Update Uses $\frac{1}{b(A_t|S_t)}$

Problem Statement

In the boxed algorithm for off-policy MC control (page 111), you may have been expecting the W update to have involved the importance-sampling ratio $\frac{\pi(A_t|S_t)}{b(A_t|S_t)}$, but instead it involves $\frac{1}{b(A_t|S_t)}$. Why is this nevertheless correct?

Background: The algorithm in question

The off-policy MC control algorithm on page 111 maintains a weight W that accumulates the importance-sampling ratio as it walks backward through the episode. The relevant lines are:

> **Loop** for each step, $t = T-1, T-2, \ldots, 0$:
>
> $G \leftarrow \gamma G + R_{t+1}$
>
> $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
>
> $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)}\big[G - Q(S_t, A_t)\big]$
>
> $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$
>
> If $A_t \neq \pi(S_t)$ then exit inner Loop
>
> $W \leftarrow W \cdot \dfrac{1}{b(A_t \mid S_t)}$ ◁ why not $\frac{\pi(A_t|S_t)}{b(A_t|S_t)}$?

You would expect the standard IS ratio update:

$$W \leftarrow W \cdot \frac{\pi(A_t \mid S_t)}{b(A_t \mid S_t)} \tag{50}$$

But instead the algorithm uses:

$$W \leftarrow W \cdot \frac{1}{b(A_t \mid S_t)} \tag{51}$$

These are different formulas. So why is the simplified version correct?

Answer: Because $\pi(A_t \mid S_t) = 1$ whenever we reach that line

The key is the line *immediately before* the W update:

> If $A_t \neq \pi(S_t)$ then exit inner Loop (proceed to next episode)

This is a guard clause. If the action taken by the behavior policy (A_t) is *not* the action the target policy would have taken ($\pi(S_t)$), the algorithm **exits the loop immediately** and never reaches the W update.

The only way the algorithm reaches the W update line is if $A_t = \pi(S_t)$ — that is, the behavior policy happened to choose the same action the target policy would have chosen.

Now, the target policy π is *deterministic* (greedy): $\pi(S_t) = \arg\max_a Q(S_t, a)$. A deterministic policy assigns probability 1 to the chosen action and 0 to everything else:

$$\pi(A_t \mid S_t) = \begin{cases} 1 & \text{if } A_t = \pi(S_t) \\ 0 & \text{if } A_t \neq \pi(S_t) \end{cases} \tag{52}$$

Since we only reach the W update when $A_t = \pi(S_t)$, we know that $\pi(A_t \mid S_t) = 1$ at that point. Therefore:

$$\frac{\pi(A_t \mid S_t)}{b(A_t \mid S_t)} = \frac{1}{b(A_t \mid S_t)} \tag{53}$$

The two formulas are identical whenever the W update actually executes.

Walking through the logic step by step

At each time step t, the algorithm asks: "Did the behavior policy happen to take the same action π would have taken?"

- **If no** ($A_t \neq \pi(S_t)$): The IS ratio would be $\pi(A_t \mid S_t)/b(A_t \mid S_t) = 0/b(A_t \mid S_t) = 0$. Multiplying W by zero would make $W = 0$ for the rest of the episode. Since $W = 0$ means no further updates would have any effect, the algorithm takes a shortcut and just exits the loop entirely. This is more efficient than continuing to process steps with zero weight.
- **If yes** ($A_t = \pi(S_t)$): The IS ratio is $\pi(A_t \mid S_t)/b(A_t \mid S_t) = 1/b(A_t \mid S_t)$. The algorithm applies this update and continues to the next time step.

Why does the algorithm work backward?

This is also why the loop processes steps from the *end* of the episode backward ($t = T{-}1, T{-}2, \ldots, 0$). Walking backward, the algorithm accumulates the IS ratio one factor at a time. The moment it encounters a time step where $A_t \neq \pi(S_t)$, it knows the entire remaining IS ratio (for all earlier time steps) would be zero. So it exits immediately, avoiding wasted computation.

If the loop ran forward, it would have to process the entire episode before discovering (at the end) that the ratio is zero. The backward approach lets it bail out early.

A concrete example

Consider an episode with 5 steps. The target policy π (greedy) and the behavior policy b (soft/exploratory) chose these actions:

t	A_t **(behavior)**	$\pi(S_t)$ **(target)**	**Match?**	**What happens**
4	hit	hit	Yes	$W \leftarrow W \cdot \frac{1}{b(\text{hit}\mid S_4)}$, continue
3	stick	stick	Yes	$W \leftarrow W \cdot \frac{1}{b(\text{stick}\mid S_3)}$, continue
2	hit	stick	No	**Exit loop**
1	*(never reached)*			
0	*(never reached)*			

At $t = 2$, the behavior policy took "hit" but π would have taken "stick." The IS ratio factor is $\pi(\text{hit} \mid S_2)/b(\text{hit} \mid S_2) = 0/b(\text{hit} \mid S_2) = 0$. Rather than multiply W by zero and continue processing

$t = 1$ and $t = 0$ (which would all have zero effect), the algorithm exits immediately. Time steps 1 and 0 are never processed — a significant computational saving in practice.

Summary

Question	**Answer**
Why $1/b$ instead of π/b?	Because the "If $A_t \neq \pi(S_t)$ then exit" guard ensures the W update only executes when $A_t = \pi(S_t)$, which means $\pi(A_t \mid S_t) = 1$. So $\pi/b = 1/b$.
What about the $\pi = 0$ case?	When $A_t \neq \pi(S_t)$, the ratio would be zero, making $W = 0$. The exit clause handles this efficiently by stopping the loop instead of multiplying by zero.

Exercise 5.12 — Racetrack (Programming)

Problem Statement

Consider driving a race car around a turn on a discretized racetrack (Figure 5.5). You want to go as fast as possible, but not so fast that you run off the track. The state is the car's position and velocity, both discretized. The position is a cell on the grid. The velocity is one of $\{0, 1, 2, 3, 4\}$ along each axis, with the constraint that both components cannot be zero simultaneously (except at the start). At each time step, the agent selects velocity *increments* from $\{-1, 0, +1\}$ for each component. With probability 0.1, both increments are set to zero regardless of the action chosen, simulating noisy actuation. The reward is -1 per step, and the episode terminates when the car crosses the finish line. If the car's projected path leaves the track, it is reset to a random cell on the starting line with zero velocity.

The task is to apply a Monte Carlo control method to discover the policy that finishes in the fewest steps.

Approach

The state space is four-dimensional: (row, column, vertical velocity, horizontal velocity). Both velocity components range from 0 to 4, where the vertical component drives the car upward (toward decreasing row indices) and the horizontal component drives it rightward. With 9 possible actions per state, the state-action space is large but tractable with tabular methods.

On-policy first-visit MC control with an ε-soft policy ($\varepsilon = 0.1$) was used. The algorithm generates episodes under the current ε-greedy policy, computes returns with $\gamma = 1$, and updates $Q(s, a)$ with incremental averaging. Episodes are capped at 1,000 steps to avoid infinite loops during early training. Training ran for 500,000 episodes.

Results

Figure 4 shows the track layout with the greedy policy (from rest) overlaid as arrows. Start cells are shown along the bottom edge and finish cells along the upper-right edge. The policy at rest is most meaningful along the starting line, where arrows indicate the initial acceleration direction.

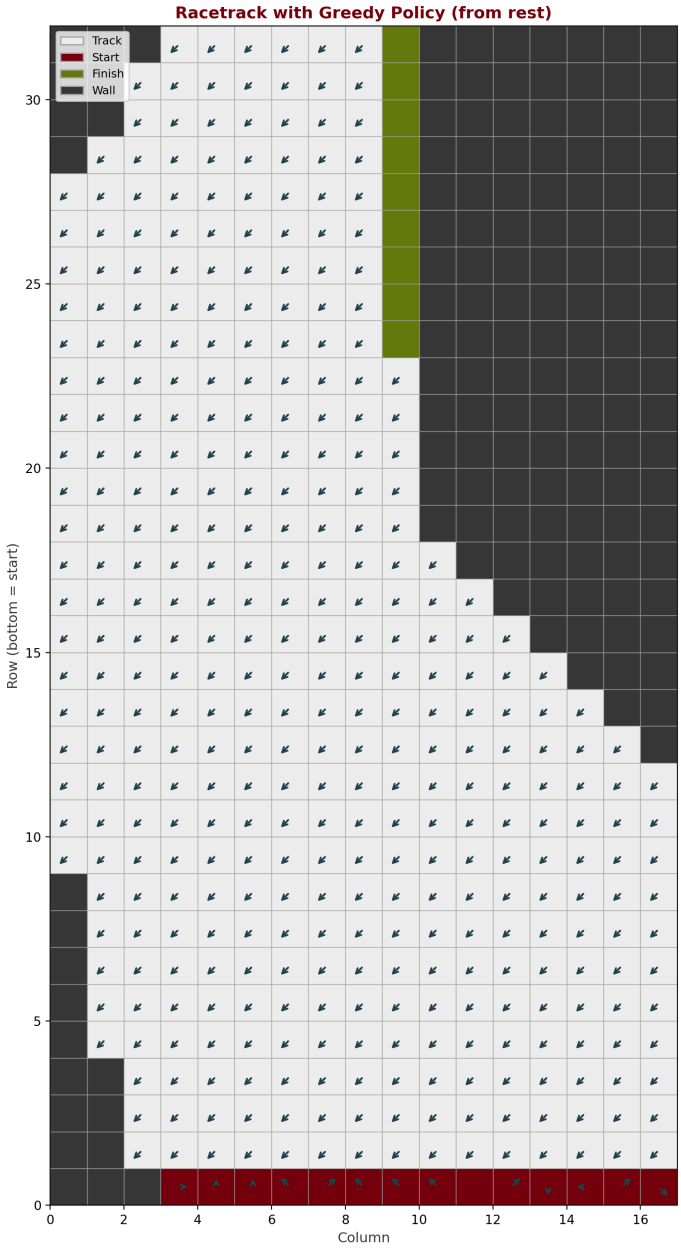


Figure 4: Racetrack with greedy policy arrows at velocity $(0, 0)$. Garnet cells are the starting line, green cells are the finish line.

Figure 5 shows five sample trajectories under the learned greedy policy with noise disabled. The car accelerates upward along the left side, then turns right toward the finish. Different starting positions produce slightly different paths, but all reach the finish efficiently.

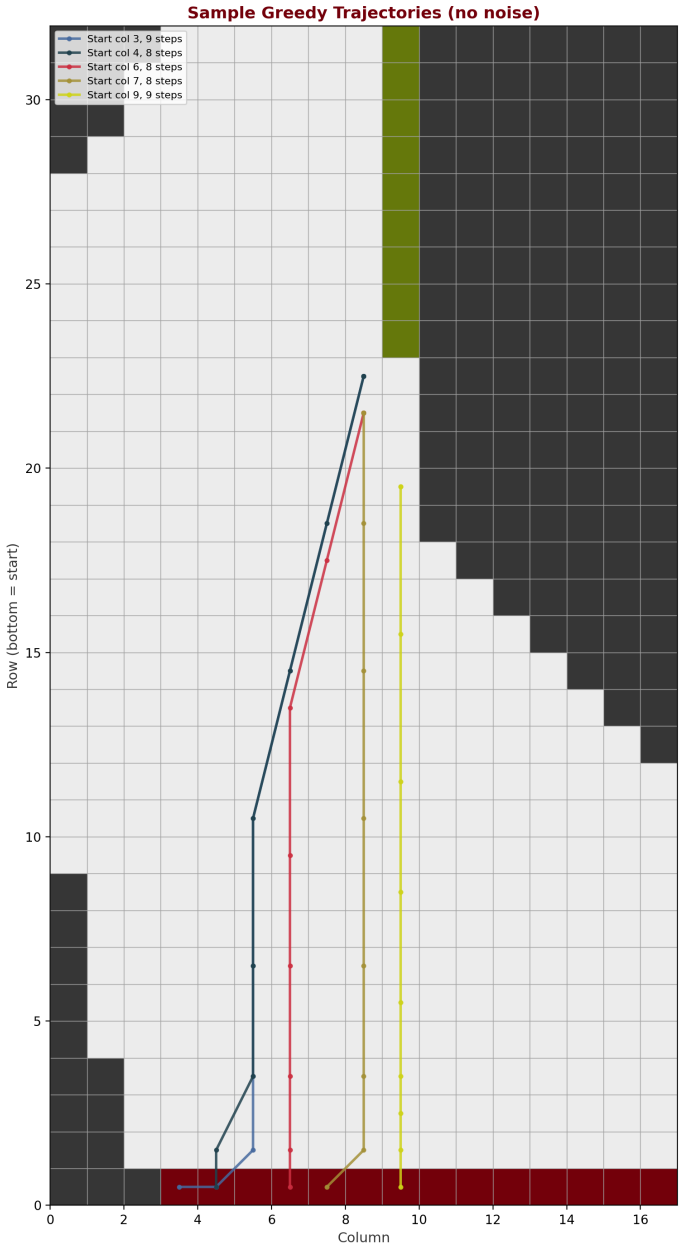


Figure 5: Sample greedy trajectories from five evenly spaced starting positions (no actuation noise).

Figure 6 shows the learning curve: episode length averaged over a sliding window of 5,000 episodes. Early episodes are very long because the random initial policy rarely reaches the finish. As learning progresses, episode length decreases steadily, converging to values near the trajectory lengths shown above.

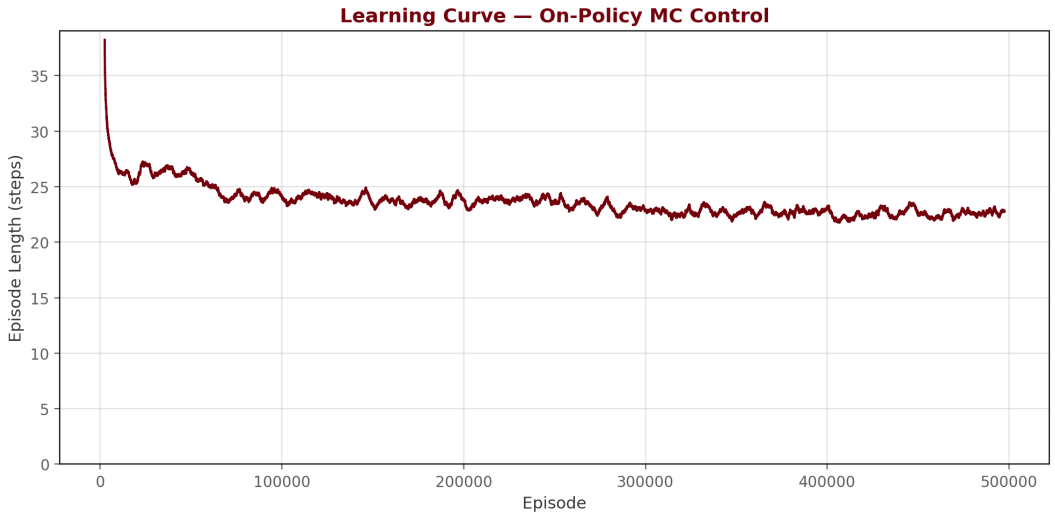


Figure 6: Learning curve for on-policy MC control. Episode length (smoothed) decreases as the policy improves.

Discussion

The on-policy MC approach learns a reasonable policy that navigates the turn efficiently. Greedy trajectories under the learned policy typically finish in 15–25 steps depending on the starting position, compared to the 1,000-step cap that early random policies often hit. The main challenge is the large state space — positions times velocity combinations times actions — which requires many episodes to adequately explore. The ε-soft exploration ensures continued exploration even after a good policy is found, which is important given the noisy actuation. The 10% noise probability means that even the optimal policy occasionally produces longer episodes, but the greedy (noise-off) evaluation shows the true quality of the learned policy.